Manual de Infraestructura Linux

I. Montando la estructura de carpetas 1
II. Usuarios y Grupos (Gestión de ASIR) 3
III. Políticas de Seguridad y Permisos Avanzados 8
IV. Auditoría, Monitorización y Mantenimiento 9
V. Prácticas de Consolidación 11
VI. Seguridad Granular: Listas de Control de Acceso (ACL) 12

Hecho por: Carlos Parriego Urbano**.**

Curso: ASIR 1º
Asignatura: Implantación de Sistemas Operativos.

I. Montando la estructura de carpetas

1.1. Crear los directorios de golpe `sudo mkdir -p /var/corporativos/{politicas, presupuestos}/proyectos/{dev, mark, ops}`

El uso de `sudo` es obligatorio aquí porque estamos trabajando sobre la ruta `/var`, un directorio del sistema donde los usuarios sin privilegios no tienen permisos de escritura. El parámetro `-p` (parents) es fundamental para la eficiencia: le indica al sistema que cree toda la ruta de carpetas intermedias de una sola vez. Si no lo usáramos, Linux nos daría un error. El uso de las llaves `{}` se denomina expansión de llaves. Nos permite generar múltiples directorios hermanos en la misma línea de comandos, ahorrándonos tener que escribir el comando `mkdir` cinco veces.

```
parry@srv-carlos:~$ sudo mkdir -p /var/corporativos/{politicas, presupuestos}/pro
yectos/{dev, mark, ops}
[sudo] password for parry:
```

1.2. Crear los archivos de prueba (README y demás)

```
echo "Documentacion v1" | sudo tee /var/proyectos/dev/README.md

echo "Guia de infraestructura" | sudo tee /var/proyectos/ops/README.md
echo "Estrategia de campaña" | sudo tee
/var/proyectos/mark/campania_2026.txt

echo "Manual de usuario" | sudo tee
/var/corporativos/politicas/manual.txt

echo "Presupuesto 2026: 150€" | sudo tee
/var/corporativos/presupuestos/presupuesto.txt
```

El comando `echo` simplemente imprime la cadena de texto proporcionada. Sin embargo, para escribir ese texto en un archivo protegido dentro de `/var`, no podemos usar la redirección normal (`>`) con `sudo`. Para solucionar esto, pasamos la salida de `echo` a través de una tubería (`|`) hacia el comando `tee`. Al ejecutar `sudo tee`, este comando lee la entrada estándar y la escribe en el archivo de destino con permisos de superusuario, superando la restricción de permisos.

```
parry@srv-carlos:~$ echo "Documentacion v1" | sudo tee /var/proyectos/dev/README.
md
Documentacion v1
parry@srv-carlos:~$ echo "Guia de infraestructura" | sudo tee /var/proyectos/ops/
README.md
Guia de infraestructura
parry@srv-carlos:~$ echo "Estrategia de campaña" | sudo tee /var/proyectos/mark/c
ampania_2026.txt
Estrategia de campaña
parry@srv-carlos:~$ echo "Manual de usuario" | sudo tee /var/corporativos/politic
as/manual.txt
Manual de usuario
parry@srv-carlos:~$ echo "Presupuesto 2026: 150€" | sudo tee /var/corporativos/pr
esupuestos/presupuesto.txt
Presupuesto 2026: 150€
```

1.3. Comprobación de la estructura recursiva

Una vez montada la infraestructura, necesitamos auditar que todo se ha creado correctamente. La opción `-l` nos muestra un formato de lista larga, revelando detalles cruciales como los permisos, el propietario, el grupo y la fecha de creación. La opción `-R` (recursivo) es la pieza clave: hace que el listado no se detenga en el primer nivel, sino que se adentre en cada subcarpeta para mostrar todo el árbol de directorios de forma jerárquica.

```
parry@srv-carlos:~$ ls -lR /var/proyectos /var/corporativos/
/var/corporativos/:
total 12
drwxr-xr-x 2 root root 4096 mar 10 11:44 {politicas,
drwxr-xr-x 2 root root 4096 feb 19 08:52 politicas
drwxr-xr-x 2 root root 4096 feb 19 08:53 presupuestos

/var/corporativos/{politicas,:
total 0

/var/corporativos/politicas:
total 4
-rw-r--r-- 1 root root 18 mar 10 11:47 manual.txt

/var/corporativos/presupuestos:
total 4
-rw-r--r-- 1 root root 25 mar 10 11:47 presupuesto.txt

/var/proyectos:
total 24
drwxrwxr-x+ 4 alicia  dev   4096 feb 23 11:45  dev
drwxrwsr-x  2 root    dev   4096 mar  2 13:04  devops_compartido
drwxrwx---  2 eva     mark  4096 feb 19 08:51  mark
drwxrwx---  2 antonio ops   4096 feb 19 08:49  ops
drwxr-xr-x  2     770 qa    4096 feb 26 09:49  qa
drwxrwxr-x  2 root    dev   4096 feb 23 11:51 'special project'

/var/proyectos/dev:
total 12
drwxrwxr-x  2 alicia dev 4096 feb 23 11:45 private
-rw-rw----+ 1 alicia dev   17 mar 10 11:47 README.md
drwxrwxr-x  2 alicia dev 4096 feb 23 11:44 shared

/var/proyectos/dev/private:
total 0

/var/proyectos/dev/shared:
total 0
-rwxrwxr-x 1 alicia dev 0 feb 23 11:44 test.txt

/var/proyectos/devops_compartido:
total 0
-rw-rw-r-- 1 maria dev 0 mar  2 13:04 ops_archivo.txt
ls: cannot open directory '/var/proyectos/mark': Permission denied
ls: cannot open directory '/var/proyectos/ops': Permission denied

/var/proyectos/qa:
total 0

'/var/proyectos/special project':
total 0
-rw-r--r-- 1 ana dev 0 feb 23 11:54 archivo_ana.txt
```

II. Usuarios y Grupos (Gestión de ASIR)

2.1. Crear los grupos del trabajo

```
sudo groupadd dev && sudo groupadd ops && sudo groupadd mark && sudo
groupadd ejecutivos && sudo groupadd admins
```

```
parry@srv-carlos:~$ sudo groupadd dev && sudo groupadd ops && sudo groupadd mark && sudo gr
oupadd ejecutivos && sudo groupadd admins
groupadd: group 'dev' already exists
```

```
cat /etc/group | grep -E "dev|ops|mark|ejecutivos|admins"
```

```
parry@srv-carlos:~$ cat /etc/group | grep -E "dev|ops|mark|ejecutivos|admins"
plugdev:x:46:parry
dev:x:1010:alicia,pedro,juan,carlos,Laura,paco,maria,jesse
ops:x:1011:antonio,carlos,maria
mark:x:1012:eva,fran
ejecutivos:x:1013:paco,alicia,jesse
admins:x:1014:
```

Primero definimos la estructura organizativa creando los grupos correspondientes con `groupadd`. El uso del operador `&&` permite encadenar los comandos para que se ejecuten secuencialmente. Posteriormente, utilizamos `cat` para leer el archivo `/etc/group` y lo filtramos mediante `grep -E`. Esto nos sirve como comprobación inmediata para verificar que los grupos se han creado correctamente y para visualizar el GID (Group ID) que el sistema les ha asignado automáticamente.

(A mí me sale que ya existe debido a que ya lo he hecho antes)

2.2. Crear todos los usuarios de la lista

```
sudo useradd [nombre] -m -s /bin/bash
```

```
for user in alicia pedro juan antonio carlos eva fran paco; do echo "$user:password" | sudo chpasswd; done
```

Para la creación de perfiles usamos `useradd` con dos flags esenciales: `-m` para forzar la creación del directorio personal del usuario en `/home`; y `-s /bin/bash` para asignarles una shell interactiva y completa. Para agilizar el trabajo, aplicamos un bucle `for` de Bash que recorre la lista de usuarios. Dentro del bucle, generamos una cadena y se la pasamos a `chpasswd`. Esto nos permite establecer una contraseña genérica ("password") para todos simultáneamente.

```
parry@srv-carlos:~$ sudo useradd [nombre] -m -s /bin/bash
parry@srv-carlos:~$ for user in alicia pedro juan antonio carlos eva fran paco; do echo "$u
ser:password" | sudo chpasswd; done
parry@srv-carlos:~$ |
```

2.3. Asignación masiva de contraseñas

```
for user in alicia pedro juan antonio carlos eva fran paco; do echo "$user:password" | sudo chpasswd; done
```

```
cut -d: -f1 /etc/passwd | grep -E "alicia|pedro|juan|antonio|carlos|eva|fran|paco"
```

Con el bucle `for` ponemos la misma contraseña ("password") a todos de una vez. Usamos `cut -d: -f1` sobre el archivo `/etc/passwd` para aislar el primer campo (los nombres de usuario) y confirmamos mediante la tubería hacia `grep` que los perfiles han sido registrados correctamente en la base de datos de usuarios del sistema.

```
parry@srv-carlos:~$ for user in alicia pedro juan antonio carlos eva fran paco; do echo "$u
ser:password" | sudo chpasswd; done
[sudo] password for parry:
parry@srv-carlos:~$ cut -d: -f1 /etc/passwd | grep -E "alicia|pedro|juan|antonio|carlos|eva
|fran|paco"
alicia
pedro
antonio
carlos
eva
fran
paco
juan
```

2.4. Asignación de usuarios a sus respectivos grupos

```
sudo usermod -aG dev alicia
sudo usermod -aG dev pedro
sudo usermod -aG dev juan
sudo usermod -aG ops antonio
sudo usermod -aG ops carlos
sudo usermod -aG mark eva
sudo usermod -aG mark fran
sudo usermod -aG ejectutivos paco
sudo usermod -aG ejectutivos alicia
```

Para incluir a los usuarios en sus departamentos utilizamos `usermod`. Es de vital importancia usar la combinación `-aG`. La `-G` indica los grupos secundarios, pero la `-a` (append) asegura que el usuario se añada a esos nuevos grupos sin ser eliminado de los grupos a los que ya pertenecía previamente.

```
parry@srv-carlos:~$ sudo usermod -aG dev alicia
parry@srv-carlos:~$ sudo usermod -aG dev pedro
parry@srv-carlos:~$ sudo usermod -aG dev juan
parry@srv-carlos:~$ sudo usermod -aG ops antonio
parry@srv-carlos:~$ sudo usermod -aG ops carlos
parry@srv-carlos:~$ sudo usermod -aG mark eva
parry@srv-carlos:~$ sudo usermod -aG mark fran
parry@srv-carlos:~$ sudo usermod -aG ejecutivos paco
parry@srv-carlos:~$ sudo usermod -aG ejecutivos alicia
```

2.5. Verificación de identidad y grupos

```
id alicia

groups alicia
```

Para validar el paso anterior tenemos dos herramientas: `id`, que nos proporciona una salida detallada incluyendo el UID (User ID), el GID del grupo principal y los IDs de los grupos secundarios; y `groups`, que nos da la misma información pero de una manera más limpia y resumida (solo los nombres de los grupos).

```
parry@srv-carlos:~$ id alicia
uid=1008(alicia) gid=1015(alicia) groups=1015(alicia),1010(dev),1013(ejecutivos)
parry@srv-carlos:~$ groups alicia
alicia : alicia dev ejecutivos
```

2.6. Creación de usuarios con parámetros específicos

```
sudo useradd -m -s /bin/bash -c "Desarollador Senio" saul
sudo useradd -m -s /bin/bash -g dev gustavo
sudo useradd -m -s /bin/bash -G dev,ejecutivos jesse
```

Aquí realizamos una gestión de usuarios mucho más granular. El parámetro `-c` modifica el campo GECOS, permitiendo añadir comentarios útiles como el cargo del empleado. El parámetro `-g` fuerza que el grupo principal del usuario sea uno específico. El parámetro `-G` permite asignar al usuario a múltiples grupos secundarios desde el momento de su creación, separándolos por comas.

```
parry@srv-carlos:~$ sudo useradd -m -s /bin/bash -c "Desarollador Senio" saul
parry@srv-carlos:~$ sudo useradd -m -s /bin/bash -g dev gustavo
useradd: user 'gustavo' already exists
parry@srv-carlos:~$ sudo useradd -m -s /bin/bash -G dev,ejecutivos jesse
useradd: user 'jesse' already exists
```

Already exist = significa que ya existe, puesto que lo he creado antes.

2.7. Modificación de perfiles y bloqueo de cuentas

```
sudo usermod -s /bin/sh juan
sudo usermod -L juan
sudo usermod -U juan
sudo usermod -l prueba juan
```

A través de `usermod` gestionamos el ciclo de vida de la cuenta. Cambiamos la shell de acceso con `-s`. El bloqueo de la cuenta se realiza con `-L` (Lock), lo que impide temporalmente que el usuario inicie sesión. La cuenta se restaura a su estado activo con `-U` (Unlock). Finalmente, la opción `-l` (login) permite renombrar por completo el nombre de usuario de inicio de sesión de "juan" a "prueba".

```
parry@srv-carlos:~$ sudo usermod -s /bin/sh juan
parry@srv-carlos:~$ sudo usermod -L juan
parry@srv-carlos:~$ sudo usermod -U juan
parry@srv-carlos:~$ sudo usermod -l prueba juan
```

2.8. Eliminación de usuarios y limpieza de rastro

`sudo userdel -r saul`

Cuando un empleado causa baja, no basta con borrar su perfil. El uso del parámetro `-r` (remove) junto a `userdel` es clave: garantiza que, además de borrar el usuario, se elimine por completo su directorio personal (`/home/saul`). Esto evita la acumulación de archivos huérfanos, optimiza el espacio en disco y mantiene la seguridad del servidor.

```
parry@srv-carlos:~$ sudo userdel -r saul
```

2.9. Gestión avanzada de grupos (GID y nombres)

`sudo groupadd -g 2050 auditores`

`sudo groupmod -n otraprueba prueba`

`sudo groupdel otraprueba`

Para la administración de grupos, podemos forzar un ID numérico específico (GID) usando la bandera `-g`, lo cual es muy útil para mantener la consistencia entre servidores. Modificamos nombres de grupos con `groupmod -n` y los eliminamos definitivamente del sistema con `groupdel`.

```
parry@srv-carlos:~$ sudo groupadd -g 2050 auditores
groupadd: group 'auditores' already exists
parry@srv-carlos:~$ sudo groupmod -n otraprueba prueba
groupmod: group 'prueba' does not exist
parry@srv-carlos:~$ sudo groupdel otraprueba
groupdel: group 'otraprueba' does not exist
```

Aquí me sale que existe y no existe porque como ya lo he hecho antes, está intentando crear una cosa que ya estaba antes y está intentando borrar una cosa que he borrado antes.

2.10. Auditoría de grupos y contraseñas

```
getent group dev
sudo passwd -S juan
sudo chage -l juan
```

Para las labores de auditoría: `getent group` consulta la base de datos de grupos mostrando todos los miembros actuales (es más robusto que un simple `cat`). `passwd -S` muestra un reporte rápido del estado de la contraseña de un usuario. Por último, `chage -l` muestra las políticas de caducidad y envejecimiento de la cuenta, indicando los plazos forzosos de cambio de clave.

```
parry@srv-carlos:~$ getent group dev
dev:x:1010:alicia,pedro,carlos,Laura,paco,maria,jesse,prueba
parry@srv-carlos:~$ sudo passwd -S juan
passwd: user 'juan' does not exist
parry@srv-carlos:~$ sudo chage -l juan
chage: user 'juan' does not exist in /etc/passwd
```

III. Políticas de Seguridad y Permisos Avanzados

3.1. Permisos tradicionales y cambios de propietario

```
sudo chmod 660 /var/proyectos/dev/README.md
sudo chown alicia:dev /var/proyectos/dev/README.md
```

Implementamos seguridad a nivel de archivo usando notación octal. El permiso `660` se traduce en: el propietario (6 = lectura 4 + escritura 2) y el grupo propietario (6 = lectura 4 + escritura 2) tienen control para modificar el archivo. El último `0` garantiza que el resto de los usuarios ("otros") no tengan ningún acceso. Luego, con `chown usuario:grupo`, establecemos explícitamente a Alicia como dueña y a `dev` como el grupo asociado.

```
parry@srv-carlos:~$ sudo chmod 660 /var/proyectos/dev/README.md
parry@srv-carlos:~$ sudo chown alicia:dev /var/proyectos/dev/README.md
```

3.2. Configuración del bit SGID para trabajo colaborativo

```
sudo chmod 2775 /var/proyectos/ops/builds
```

En entornos colaborativos, el número 2 inicial en la notación octal activa el bit especial **SGID** (Set Group ID) sobre el directorio. Esto altera el comportamiento por defecto de Linux: cualquier archivo nuevo creado dentro de `builds` heredará automáticamente el grupo del directorio padre (`ops`). Así garantizamos que todos los miembros del equipo puedan editar el trabajo de sus compañeros.

```
parry@srv-carlos:~$ sudo chmod 2775 /var/proyectos/ops/builds
chmod: cannot access '/var/proyectos/ops/builds': No such file or directory
```

3.3. Configuración del bit SUID para ejecutables

```
 sudo bash -c ‘cat > /usr/local/bin/deploy.sh << “EOF”
> #!/bin/bash
> echo "Desplegando aplicacion como $(whoami)..."
> date >> /var/log/prueba_log.txt
> echo "Usuario $USER a comenzado el despligue" >>
/var/log/deployment.log
> EOF'

 sudo chmod +x /usr/local/bin/deploy.sh
 sudo chown rooot:root /usr/local/bin/deploy.sh
```

```
parry@srv-carlos:~$ sudo bash -c ‘cat > /usr/local/bin/deploy.sh << “EOF”
> #!/bin/bash
> echo "Desplegando aplicacion como $(whoami)..."
> date >> /var/log/prueba_log.txt
> echo "Usuario $USER a comenzado el despligue" >> /var/log/deployment.log
> EOF'
> ^C
parry@srv-carlos:~$ sudo chmod +x /usr/local/bin/deploy.sh
parry@srv-carlos:~$ sudo chown rooot:root /usr/local/bin/deploy.sh
chown: invalid user: ‘rooot:root’
parry@srv-carlos:~$ sudo chown root:root /usr/local/bin/deploy.sh
parry@srv-carlos:~$ sudo chmod 4755 /usr/local/bin/deploy.sh
parry@srv-carlos:~$
```

```
sudo chmod 4755 /usr/local/bin/deploy.sh
```

El número 4 al inicio activa el bit **SUID** (Set User ID). Esto provoca que, cuando un usuario normal lance el script, el proceso adquiera temporalmente los permisos del propietario del archivo (en este caso, `root`). Es útil para delegar tareas de administración a usuarios sin darles la contraseña de administrador, aunque debe usarse con precaución.

```
parry@srv-carlos:~$ sudo chmod 4755 /usr/local/bin/deploy.sh
parry@srv-carlos:~$
```

3.4. Configuración del Sticky Bit para seguridad en borrado

```
sudo chmod 1777 /var/proyectos/sticky_bit
```

Para crear buzones de intercambio seguro usamos el **Sticky Bit**, representado por el 1 inicial, junto con permisos 777. Creamos una carpeta pública, pero el Sticky Bit impone una restricción de seguridad clave: dentro de ese directorio compartido, un usuario solo podrá borrar o renombrar los archivos de los que él mismo sea propietario, protegiendo el trabajo ajeno de borrados accidentales.

IV. Auditoría, Monitorización y Mantenimiento

4.1. Búsqueda de archivos por propietario

```
sudo find /var/proyectos/ -user alicia
```

La herramienta `find` es indispensable para auditorías. Utilizando el parámetro `-user`, le indicamos que rastree de forma recursiva todo el directorio `/var/proyectos/` y nos devuelva únicamente los archivos que le pertenecen a Alicia. Esto es vital para controlar cuotas o localizar archivos dispersos antes de borrar una cuenta.

```
parry@srv-carlos:~$ sudo find /var/proyectos/ -user alicia
/var/proyectos/dev
/var/proyectos/dev/private
/var/proyectos/dev/README.md
/var/proyectos/dev/shared
/var/proyectos/dev/shared/test.txt
```

4.2. Filtrado de usuarios reales del sistema

```
awk -F: '$3 >= 1000 {print $1, "UID=" $3}' /etc/passwd
```

Para obtener un informe limpio de los usuarios humanos, procesamos `/etc/passwd` con `awk`. La condición `$3>=1000` filtra la tercera columna (el UID). En Linux, los UIDs menores a 1000 están reservados para demonios internos del sistema. Al exigir que sea 1000 o superior, filtramos la "basura" y nos quedamos solo con las cuentas reales, formateando la salida para ver el nombre y el UID de forma clara.

```
parry@srv-carlos:~$ awk -F: '$3 >= 1000 {print $1, "UID=" $3}' /etc/passwd
nobody UID=65534
parry UID=1000
tecnico UID=1001
tecnico1 UID=1002
tecnico2 UID=1003
sftp1 UID=1006
sftp2 UID=1007
alicia UID=1008
pedro UID=1009
antonio UID=1011
carlos UID=1012
eva UID=1013
fran UID=1014
paco UID=1015
ana UID=1016
Laura UID=1017
maria UID=1018
gustavo UID=1020
jesse UID=1021
[nombre] UID=1022
prueba UID=1010
```

4.3. Control de conexiones y sesiones activas

```
lastlog | grep -E "alicia|juan"
```

```
who
```

Monitorizar quién entra al sistema es una tarea básica de seguridad. El comando `lastlog` consulta la base de datos de inicios de sesión y nos indica la fecha exacta de la última conexión. Con `grep` filtramos para ver solo a personas de interés. Por otro lado, el comando `who` nos ofrece una radiografía en tiempo real: nos enseña qué usuarios están logueados en la máquina en ese preciso instante y desde dónde.

```
parry@srv-carlos:~$ lastlog | grep -E "alicia|juan"
alicia           pts/1    192.168.103.122                          jue feb 26 09:12:00 +00
00 2026
parry@srv-carlos:~$ who
parry    tty1         2026-03-10 11:11
parry    pts/0        2026-03-10 11:44 (192.168.103.122)
```

V. Prácticas de Consolidación

5.1. Comprobación de acceso de usuarios nuevos

```
sudo -u roberto ls -la /var/proyectos/dev/private
```

No basta con configurar los permisos; hay que probarlos. El parámetro `-u` del comando `sudo` nos permite ejecutar una instrucción asumiendo la identidad de otro usuario (Roberto), sin necesidad de conocer su contraseña. Esto nos permite comprobar empíricamente si las barreras de permisos que hemos levantado funcionan y le deniegan el paso.

```
parry@srv-carlos:~$ sudo -u roberto ls -la /var/proyectos/dev/private
sudo: unknown user roberto
sudo: error initializing audit plugin sudoers_audit
```

5.2. Aislamiento de directorios privados

```
sudo chmod 700 /var/proyectos/dev/private
```

Cuando necesitamos confidencialidad absoluta en un directorio, aplicamos el permiso `700`. Esto otorga permisos totales exclusivamente al propietario. El resto de los números en cero significan que ningún otro usuario del sistema puede ver el contenido, ni siquiera los compañeros que pertenezcan a su mismo grupo departamental.

```
parry@srv-carlos:~$ sudo chmod 700 /var/proyectos/dev/private
parry@srv-carlos:~$ |
```

VI. Auditoría, Monitorización y Mantenimiento (Continuación)

6.1 Generación de un reporte de auditoría automatizado

```
echo " == REPORTE AUDITORIA USUARIOS == " > ~/reporte_usuarios.txt

echo "Fecha:$(date)" >> ~/reporte_usuarios.txt
```

Iniciamos la creación de un archivo de texto llamado `reporte_usuarios.txt` en el directorio personal (~).

- El operador **>** crea el archivo (o lo sobrescribe si ya existe) con el título del reporte.
- El operador **>>** añade información al final del archivo sin borrar lo anterior. Aquí usamos la sustitución de comando `$(date)` para estampar la fecha y hora exacta de la auditoría.

6.2 Extracción avanzada de usuarios y sus grupos

```
awk -F: '$3 >=1000 && $3 < 65534 {print $1, "UID="$3}' /etc/passwd | while read user uid; do
echo "$user ($uid) -> grupos: $(id -Gn ${user%% *})" >> ~/reporte_usuarios.txt
```

done

Este es un comando compuesto de alta eficiencia para mapear la identidad del sistema:

1. **`awk -F: '$3 >=1000 && $3 < 65534'`**: Filtra el archivo `/etc/passwd` para obtener solo usuarios humanos (UID ≥ 1000), excluyendo al usuario `nobody` (65534).
2. **`while read user uid`**: Recibe cada línea filtrada y la procesa en un bucle.
3. **`id -Gn`**: Para cada usuario encontrado, este comando obtiene los nombres de todos los grupos a los que pertenece.
4. El resultado final se concatena en el reporte, mostrando nombre, UID y membresías de grupo de forma legible.

6.3. Auditoría de privilegios administrativos (SUDO)

```
grep '^sudo:' /etc/group >> ~/reporte_usuarios.txt
```

Utilizamos grep para buscar la línea que comienza con "sudo" en el archivo /etc/group. Esto nos permite identificar inmediatamente qué usuarios tienen permisos de superusuario en el sistema, una información crítica para la seguridad del servidor.

6.4. Verificación del estado de las contraseñas

```
for user in alicia juan roberto eva enrique; do
   status=$(sudo passwd -S $user | awk '{print $1 $2}')
   echo "$status" >> ~/reporte_usuarios.txt
done
```

Mediante un bucle for, recorremos una lista de usuarios específicos para auditar su seguridad:

- **passwd -S**: Reporta el estado de la contraseña (si está bloqueada "L", si no tiene contraseña "NP" o si es utilizable "P").
- Como se observa en la captura, si un usuario no existe (como 'juan' o 'roberto'), el sistema devuelve un error, lo que sirve para detectar cuentas que han sido eliminadas pero que aún se intentan auditar.

```
parry@srv-carlos:~$  #$ echo " == REPORTE AUDITORIA USUARIOS == " > ~/reporte_usuarios.txt
parry@srv-carlos:~$  echo "Fecha:$(date)" >> ~/reporte_usuarios.txt
parry@srv-carlos:~$  echo -e "\n --- Usuarios humanos (UID >= 1000) --- " >> ~/reporte_usuarios.txt
parry@srv-carlos:~$ awk -F: '$3 >=1000 && $3 < 65534 {print $1, "UID="$3}' /etc/passwd | while read user uid; do echo "$user ($uid) -> grupos: $(id -Gn ${user%% *})" >> ~/reporte_usuarios.txt; done
parry@srv-carlos:~$ echo -e "\n --- Usuarios con acceso a SUDO --- " >> ~/reporte_usuarios.txt
parry@srv-carlos:~$ grep '^sudo:' /etc/group >> ~/reporte_usuarios.txt
parry@srv-carlos:~$ for user in alicia juan roberto eva
> ^C
parry@srv-carlos:~$ for user in alicia juan roberto eva enrique; do status=$(sudo passwd -S $user | awk '{print $1 $2}'); echo "$status" >> ~/reporte_usuarios.txt; done
[sudo] password for parry:
```

6.5. Visualización del reporte final

cat ~/reporte_usuarios.txt

El comando cat (concatenate) se utiliza para imprimir el contenido del archivo directamente en la terminal. En nuestro flujo de trabajo, esto sirve como la **validación final de la auditoría**.

Al ejecutarlo, podemos observar tres bloques de información claramente diferenciados:

- **Encabezado y Fecha**: Confirma el momento exacto en que se ejecutó el script de monitorización.
- **Listado de Usuarios Humanos**: Muestra de forma limpia el nombre de usuario, su UID (siempre ≥ 1000) y, gracias al bucle anterior, la lista completa de sus grupos.
- **Estado de Privilegios y Contraseñas**: Permite verificar de un vistazo quién tiene acceso a sudo y el estado de actividad de las claves (marcado con "P" para las cuentas activas).

```
parry@srv-carlos:~$ cat ~/reporte_usuarios.txt

--- Usuarios humanos (UID >= 1000) ---
Fecha:jue 12 mar 2026 09:01:39 UTC

 --- Usuarios humanos (UID >= 1000) ---
parry (UID=1000) -> grupos: parry adm cdrom sudo dip plugdev lxd
tecnico (UID=1001) -> grupos: tecnico sudo users
tecnico1 (UID=1002) -> grupos: tecnico1 users soporte tecnicos
tecnico2 (UID=1003) -> grupos: tecnico2 users soporte
sftp1 (UID=1006) -> grupos: sftp1 sftpusers
sftp2 (UID=1007) -> grupos: sftp2 sftpusers
alicia (UID=1008) -> grupos: alicia dev ejecutivos
pedro (UID=1009) -> grupos: pedro dev
antonio (UID=1011) -> grupos: antonio ops
carlos (UID=1012) -> grupos: carlos dev ops
eva (UID=1013) -> grupos: eva mark qa
fran (UID=1014) -> grupos: fran mark
paco (UID=1015) -> grupos: paco dev ejecutivos
ana (UID=1016) -> grupos: dev
Laura (UID=1017) -> grupos: Laura dev
maria (UID=1018) -> grupos: maria dev ops
gustavo (UID=1020) -> grupos: dev
jesse (UID=1021) -> grupos: jesse dev ejecutivos
[nombre] (UID=1022) -> grupos: [nombre]
prueba (UID=1010) -> grupos: juan dev qa

 --- Usuarios con acceso a SUDO ---
sudo:x:27:parry,tecnico
aliciaP


evaP
```